

Data Mining practice8

Team 2

양승렬 2200409

최서현 22100133

문버리 22000245

1) 데이터 설명

- 2011/01/01 00시 ~ 2012/12/19 23시
- 시간별 데이터
- 매월 20일부터는 test data

이름	타입	설명
datetime	character	hourly date + timestamp
season	integer	1 = spring, 2 = summer, 3 = fall, 4 = winter
holiday	integer	whether the day is considered a holiday
workingday	integer	whether the day is neither a weekend nor holiday
weather	integer	1 : Clear, Few clouds, Partly cloudy, Partly cloudy 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds 4: Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog
temp	double	temperature in Celsius
atemp	double	"feels like" temperature in Celsius
humidity	integer	relative humidity
windspeed	double	wind speed
casual	integer	number of non-registered user rentals initiated
registered	integer	number of registered user rentals initiated
count	integer	number of total rentals

Practice1

- Decision Tree 모델을 사용해서 자전거 대여 수량을 예측하고, Kaggle에서 성능 평가하기

- 전처리

1. train에는 있지만 test에는 없는 변수 지우기

```
drop_features = ['casual', 'registered']

train = train.drop(drop_features, axis = 1)
```

- 모델링

```

from sklearn.tree import DecisionTreeRegressor

log_y = np.log(y_train)

cart_regressor = DecisionTreeRegressor(criterion='squared_error', random_state=42)
cart_regressor.fit(X_train, log_y)

# 예측
train_pred_cart = cart_regressor.predict(X_train)

# 평가
print(f'DT (CART) : {rmsle(log_y, train_pred_cart, True):.4f}')

```

- train 성능 : DT (CART) : 0.4868

- 캐글 제출용 파일 만들기

```

# sample 만들기
test_pred_cart = cart_regressor.predict(X_test)
sample_submission['count'] = np.exp(test_pred_cart)

sample_submission.to_csv('submission.csv', index=False)

```

- 최종 성능



submission.csv
Complete (after deadline) · 2m ago

Score: 1.62097
Private score: 1.62097

- 1번의 모델에 bagging을 적용하여 자전거 대여 수량을 예측하고, Kaggle에서 성능 평가하기

- 모델링

```

from sklearn.ensemble import BaggingRegressor

log_y = np.log(y_train)

# BaggingRegressor로 CART 모델 사용
cart_regressor = DecisionTreeRegressor(criterion='squared_error', random_state=42)
bagging_cart = BaggingRegressor(base_estimator=cart_regressor, n_estimators=10, random_state=42)
bagging_cart.fit(X_train, log_y)

# 예측
train_pred_cart_bagging = bagging_cart.predict(X_train)

# 평가
print(f'Bagging DT (CART): {rmsle(log_y, train_pred_cart_bagging, True):.4f}')

```

- train 성능 : Bagging DT (CART): 0.6388

- 최종 성능



practice2.csv

Complete (after deadline) · now

Score: 1.35340

Private score: 1.35340

Practice3

- random forest 모델을 이용해서 자전거 대여 수량을 예측하고, Kaggle에서 성능 평가하기
- feature importance 계산하여 설명하기
- 모델링

```
from sklearn.ensemble import RandomForestRegressor

# 로그 변환된 타겟값
log_y = np.log(y_train)

# RandomForest 사용
randomforest_model = RandomForestRegressor(random_state=42)
randomforest_model.fit(X_train, log_y)

# 예측
train_pred_rf = randomforest_model.predict(X_train)

# 평가
print(f'RandomForest : {rmsle(log_y, train_pred_rf, True):.4f}')
```

- train 성능 : RandomForest : 0.5916

- 최종 성능



practice3.csv

Complete (after deadline) · 2m ago

Score: 1.31379

Private score: 1.31379

- Feature importance 확인

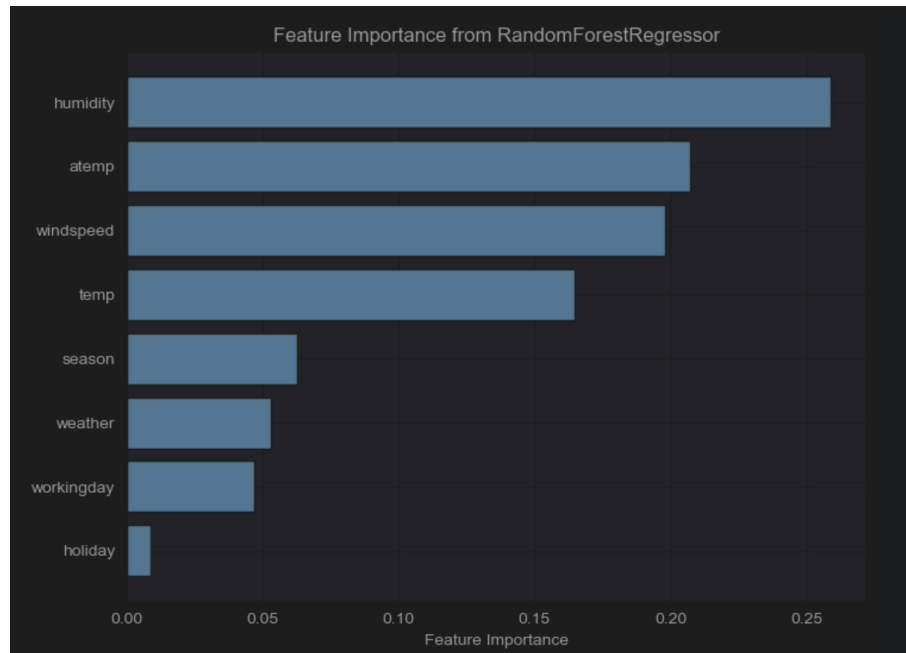
```
# Feature Importance 확인
feature_importances = randomforest_model.feature_importances_
feature_names = X_train.columns

# DataFrame으로 변환 - 내림차순 정렬
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': feature_importances
}).sort_values(by='Importance', ascending=False)

# 시각화
plt.figure(figsize=(8, 6))
plt.barh(importance_df['Feature'], importance_df['Importance'], align='center')
plt.gca().invert_yaxis()
plt.xlabel("Feature Importance")
plt.title("Feature Importance from RandomForestRegressor")
```

```
plt.show()

# 중요도 출력
for name, importance in zip(importance_df['Feature'], importance_df['Importance']):
    print(f"{name}: {importance:.4f}")
```



columns	feature importance
humidity	0.2588
atemp	0.2073
windspeed	0.1979
temp	0.1649
season	0.0626
weather	0.0528
workingday	0.0470
holiday	0.0086

Practice4

- XGBoost 모델을 이용해서 자전거 대여 수량을 예측하고, Kaggle에서 성능 평가하기
- 모델링

```
# 로그 변환된 타겟값
log_y = np.log(y_train)

# XGBoost 사용
xgboost_model = XGBRegressor(random_state=42)
xgboost_model.fit(X_train, log_y)

# 예측
train_pred_xgb = xgboost_model.predict(X_train)

# 평가
print(f'XGBoost : {rmsle(log_y, train_pred_xgb, True):.4f}')
```

- train 성능 : XGBoost : 0.9259

- 최종성능



practice4.csv

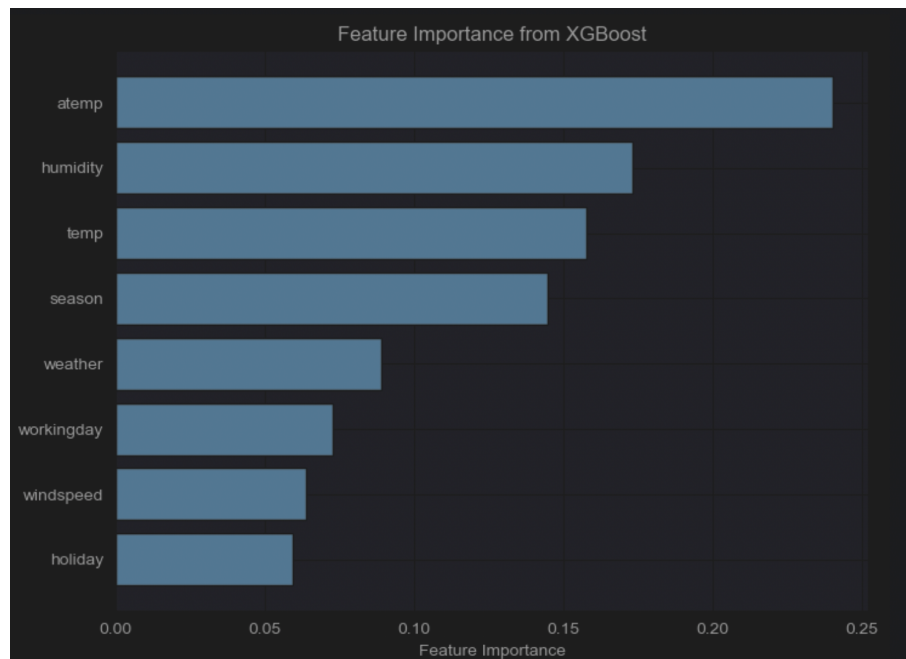
Complete (after deadline) · 1m ago

Score: 1.27274

Private score: 1.27274

- train 성능은 낮지만, test 성능은 훨씬 높은 것을 확인할 수 있음

- Feature importance



columns	feature importance
atemp	0.2400
humidity	0.1732
temp	0.1575
season	0.1446
weather	0.0890
workingday	0.0728
windspeed	0.0637
holiday	0.0592

Practice5

- Random forest나 XGBoost, Adaboost, Catboost등 여러가지 Ensemble-based model을 활용하고
- Feature engineering과 parameter tuning을 통해서 최고로 높은 성능을 얻어보자.

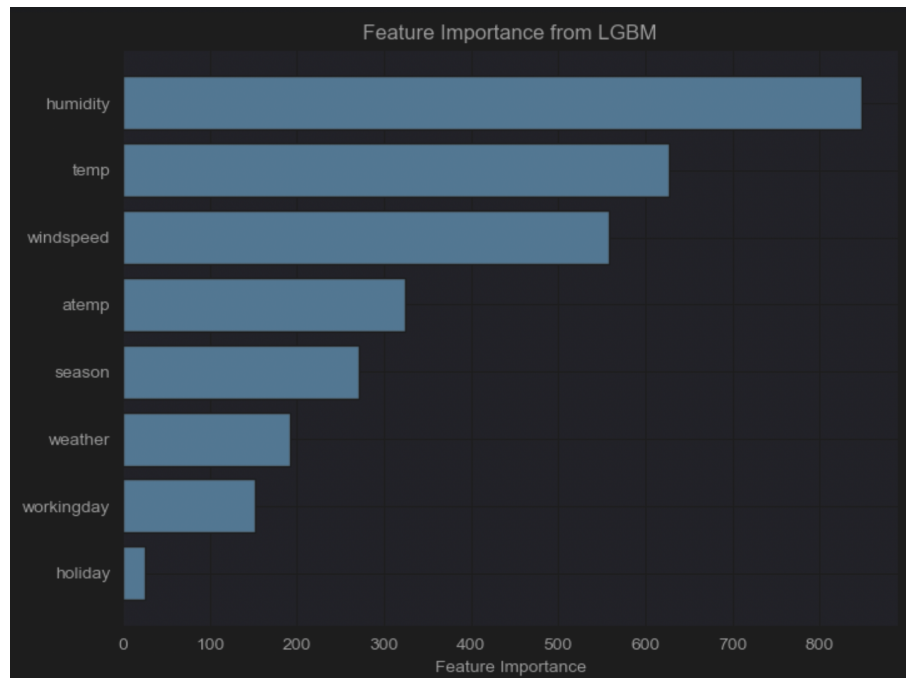
1. 모델 선정 | 다양한 모델을 활용한 베이스라인 모델링

모델 이름	train 성능	최종 성능
-------	----------	-------

Random forest	0.59	1.31
XGBoost	0.93	1.27
AdaBoost	1.26	1.26
CatBoost	1.06	1.19
LGBM	1.05	1.20

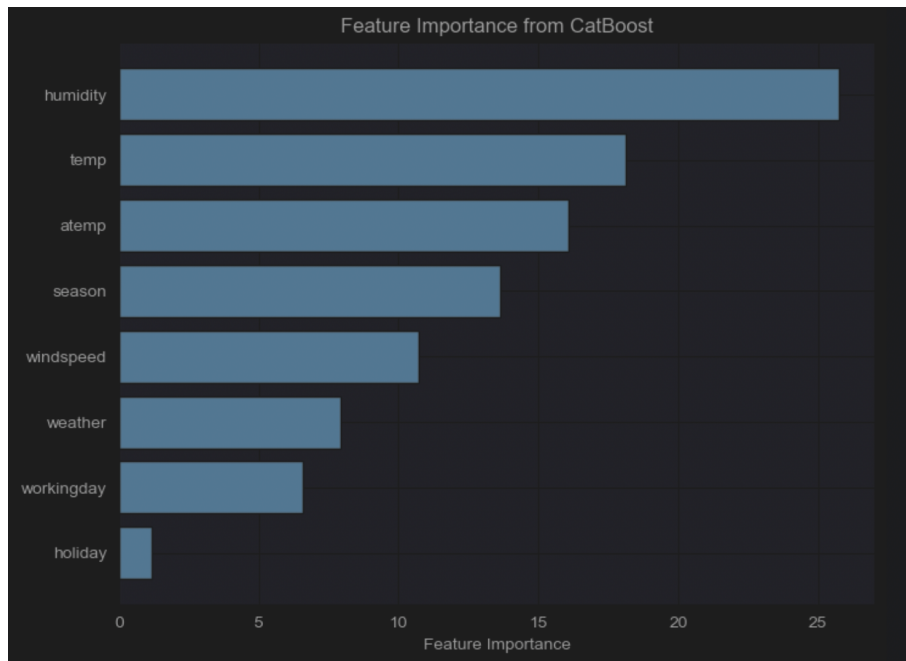
- Random Forest, XGBoost는 과적합이 있다.

▼ LGBM Feature importance



columns	feature importance
humidity	848.0000
temp	627.0000
windspeed	559.0000
atemp	325.0000
season	272.0000
weather	192.0000
workingday	152.0000
holiday	25.0000

▼ CatBoost Feature importance



columns	feature importance
humidity	25.7431
temp	18.1288
atemp	16.0880
season	13.6317
windspeed	10.7304
weather	7.9315
workingday	6.5839
holiday	1.1627

- 결론 : LGBM, CatBoost로 실험 진행

Practice5_성능 높이기

1) LGBM

	Baseline	실험1	실험2	실험3	실험4
성능 변화	1.20177	0.39918	0.38199	0.38021	0.37937

진행한 실험

1. Feature Transformation
2. Feature Interaction
3. Feature Selection
4. Grid search

[Feature engineering]

실험1 | Feature Transformation

- datetime 을 year, month, day, hour 로 split

```

### b. feature Creation
all_data['datetime'] = pd.to_datetime(all_data['datetime'], errors='coerce')

# 연도 피처 생성
all_data['year'] = all_data['datetime'].dt.year

# 월 피처 생성
all_data['month'] = all_data['datetime'].dt.month

# 시 피처 생성
all_data['hour'] = all_data['datetime'].dt.hour

# 요일 피처 생성
all_data['weekday'] = all_data['datetime'].dt.weekday

```

실험2 | Feature Interaction

- 새로운 feature interaction 추가
 - `season` 을 key로 설정 후 다양한 조합 생성
 - 계절에 따라 같은 기온이나 시간이 다르게 느껴질 수 있을 거라 판단. 예를 들어, 겨울의 15°C와 여름의 15°C는 체감이 다를 수 있고, 오후 7시가 여름과 겨울에 미치는 영향도 다를 수 있음. 이를 반영하기 위해 `season * temp` 와 `season * hour` 조합을 생성.

```

all_data["season_temp_interaction"] = all_data["season"] * all_data["temp"]
all_data["season_hour_interaction"] = all_data["season"] * all_data["hour"]

```

실험3 | Feature Selection

- Feature importance가 가장 낮은 holiday 제거
- 다중공선성 문제 해결을 위해 상관관계가 높은 변수 중 하나 제거
 - `season` VS `month`
 - `temp` VS `atemp`

```

all_data = all_data.drop(['holiday', 'month', 'temp'], axis = 1)

```

[Parameter tuning]

실험4 | Grid search

```

# 모델 생성
from sklearn.model_selection import GridSearchCV
from sklearn import metrics

# 교차 검증용 평가 함수(RMSLE 점수 계산)
rmsle_scorer = metrics.make_scorer(rmsle, greater_is_better=False)
LGBM_model = LGBMRegressor()
# 그리드서치 객체 생성
lgbm_params = {'random_state':[42], 'n_estimators':[100, 120, 140]}
gridsearch_LGBM_model = GridSearchCV(estimator=LGBM_model,
                                     param_grid=lgbm,
                                     scoring=rmsle_scorer,
                                     cv=5)

gridsearch_LGBM_model.fit(X_train, log_y)

```



```
print('최적 하이퍼파라미터 :', gridsearch_LGBM_model.best_params_)
```

[최종 성능]

LGBM_baseline.csv Complete (after deadline) · 10m	1.20177	1.20177	□
LGBM_01.csv Complete (after deadline) · 12s ago	0.39918	0.39918	□
LGBM_02.csv Complete (after deadline) · 21s ago	0.38474	0.38474	□
LGBM_03.csv Complete (after deadline) · 29s ago	0.38021	0.38021	□
LGBM_04.csv Complete (after deadline) · 35s ago	0.37937	0.37937	□

2) CatBoost

	Baseline	실험1	실험2	실험3	실험4
성능 변화	1.19	0.40101	0.39513	0.37138	0.37022

진행한 실험

1. Feature Transformation
2. Feature Selection - 1 [1차 피쳐 제거]
3. Feature Selection - 2 [다중공산성 문제 해결]
4. Parameter tuning

[Feature engineering]

실험1 | Feature Transformation

- `datetime` 을 `year`, `month`, `day`, `hour`, `weekday` 로 split

```
# 날짜 및 시간 정보 변환
train_df['datetime'] = pd.to_datetime(train_df['datetime'])
test_df['datetime'] = pd.to_datetime(test_df['datetime'])
```

```
# 연도, 월, 일, 시간, 요일 등의 새로운 컬럼 생성
for df in [train_df, test_df]:
    df['year'] = df['datetime'].dt.year
    df['month'] = df['datetime'].dt.month
    df['day'] = df['datetime'].dt.day
    df['hour'] = df['datetime'].dt.hour
    df['weekday'] = df['datetime'].dt.weekday
```

```
#test데이터에 없는 피쳐와 datetime열 제거
train_df = train_df.drop(['casual', 'registered', 'datetime'], axis = 1)
```

먼저, test 데이터에 없는 피쳐와 datetime 제거

```
# 모델 학습에 사용할 특성 및 로그 변환된 타겟 변수 정의
features = ['season', 'workingday', 'weather', 'holiday', 'temp', 'atemp',
            'windspeed', 'humidity', 'year', 'hour', 'month', 'day', 'weekday']
X = train_df[features]
y_log = np.log1p(train_df['count']) # log(1 + count) 변환

# 기본 설정의 CatBoost 모델 학습
catboost_model = CatBoostRegressor(random_seed=42)
catboost_model.fit(X, y_log)
```

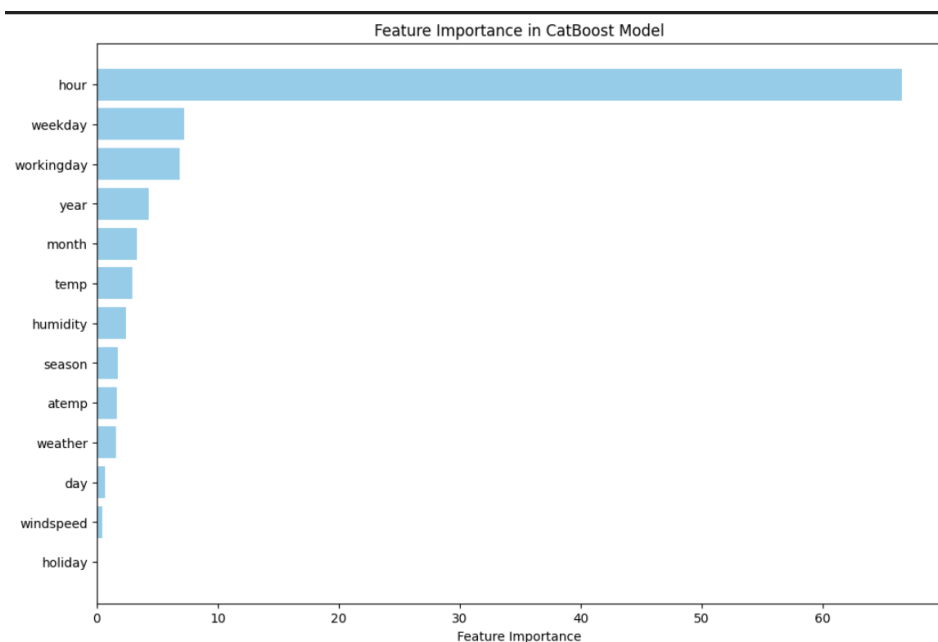
⇒ datetime에서 나온 새로운 칼럼들을 포함한 전체 피쳐 사용(base model)

Score : 0.40101

실험2 | Feature Selection - 1 [1차 피쳐 제거]

Feature Importance 계산 및 시각화

```
# Feature Importance 계산
importances = catboost_model.get_feature_importance()
feature_importances = pd.DataFrame({'Feature': features, 'Importance': importances})
feature_importances = feature_importances.sort_values(by='Importance', ascending=False)
# Feature Importance 시각화
plt.figure(figsize=(12, 8))
plt.barh(feature_importances['Feature'], feature_importances['Importance'], color='skyblue')
plt.gca().invert_yaxis() # 큰 값이 위로 오도록 설정
plt.xlabel('Feature Importance')
plt.title('Feature Importance in CatBoost Model')
plt.show()
```



hour 독립 변수가 압도적으로 CatBoost 모델에 중요한 특성인 반면, [holiday,windspeed,day]는 모델 예측에 있어 거의 영향을 주지 않음.

⇒ 모델의 복잡도를 줄이고, 일반화 성능을 높이기 위해서 중요도가 낮은 특성을 제거

Score: 0.39513

##모델 성능 향상

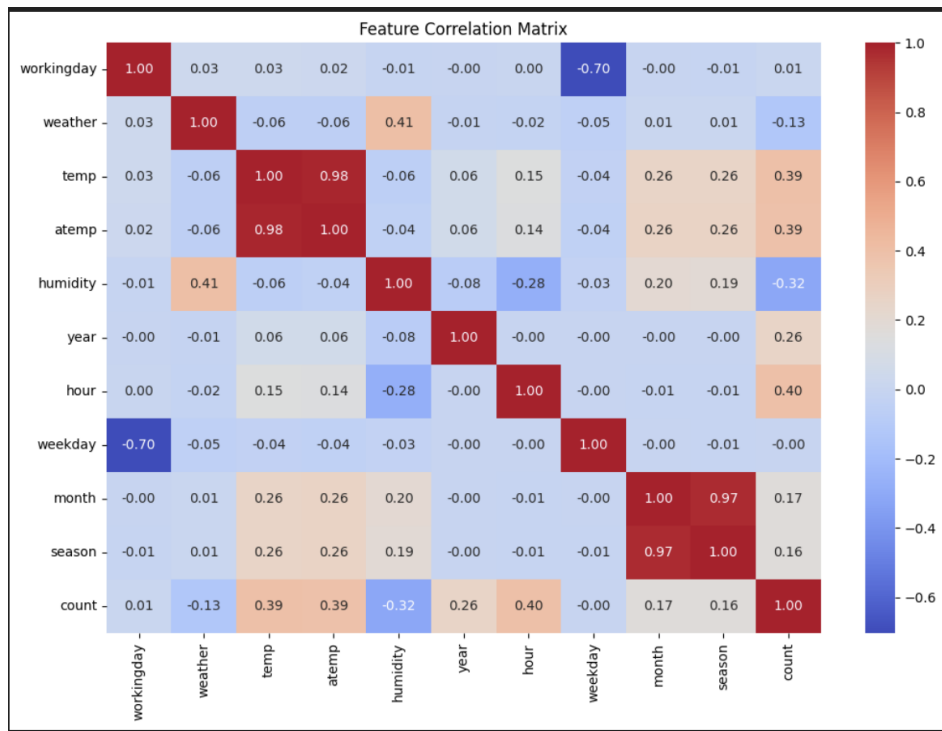
실험3 | Feature Selection - 2 [다중공선성 문제 해결]

```
import seaborn as sns
# 상관관계 행렬 계산
correlation_matrix = train_df[features + ['count']].corr()

# 상관관계 히트맵 시각화
plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
```

```
plt.title("Feature Correlation Matrix")
plt.show()
```

Correlation Matrix 활용으로 각 피쳐들의 상관계수 확인



- 상관관계가 높은 변수를 비교
 - season VS Month
 - temp VS atemp

이 모델에 어떤 피쳐가 적절한지 모든 경우의 수 계산 후 피쳐 결정

	season만 제거	Month만 제거	season,Month 제거	atemp만 제거	temp만 제거	temp,atemp제거	Month,temp 제거
score	0.42812	0.37138	0.39516	0.40261	0.40531	0.43527	0.37672

temp와 atemp의 다중공선성 문제를 해결할 수 있지만 성능을 위해 Month 피쳐만 제거.

최종 Feature Selection

Feature: ['workingday', 'weather', 'temp', 'atemp', 'humidity', 'year', 'hour', 'weekday', 'season']

Score: 0.37138

[Parameter tuning]

실험4 | Parameter tuning

```
# 기본 설정의 CatBoost 모델 학습 (전체 train 데이터를 사용)
catboost_model = CatBoostRegressor(random_seed=42, verbose=0, iterations=600,
    learning_rate=0.05, random_strength=4)
```

다양한 시도 끝에 얻은 Score : 0.36892

[최종 성능]

train rmse : 36.0376